



TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

Progetto 2: *Analisi in tempo reale di dati sui voli con Apache Flink*

Corso di Sistemi e Architetture per Big Data - A.A. 2025/26

Luca Carloni 0366574, *Alessandro Vassallo* 0366572, *Andrea Galluzzi* 0365026

Introduzione

Obiettivi

Lo studio presentato consiste nella progettazione e realizzazione di un'architettura di **stream processing**, che simuli un contesto reale di elaborazione Big Data, dalla **Data Ingestion** fino alla visualizzazione dei risultati.

Gli **obiettivi** dello studio sono volti a:

- Realizzare di tre **query** richieste dalla specifica in **Apache Flink**, realizzando anche una dashboard in Grafana per visualizzare l'andamento temporale delle metriche prodotte dalle query
- Implementare la soluzione delle tre **query** anche in **Apache Spark Streaming**.
- Implementare un **meccanismo di rilevamento di eventi fuori ordine**, confrontando diverse configurazioni di watermarking
- Implementare la gestione della **tolleranza ai guasti degli operatori con stato**

Architettura del sistema

L'architettura è composta da più servizi *containerizzati* di cui il deploy è stato effettuato tramite DockerCompose:

- *Preprocessing*
- *Producer Event Replay*
- **Apache Kafka**
- **Apache Flink**
- **Apache Spark Structured Streaming**
- **InfluxDB**
- **TimescaleDB**
- **Grafana**



Preprocessing e Producer Event Replay

Un modulo di **preprocessing** ha la responsabilità di leggere i dataset CSV mensili e arricchirli con l'**event time** di ciascun volo.

Gli eventi vengono poi ordinati temporalmente e compattati in un unico file preparato, usato come sorgente del replay.

Un **producer** dedicato riproduce il dataset storico come flusso di eventi, rispettando la temporizzazione dell'event time tramite un **fattore di accelerazione** configurabile, impostato a 57600x.

- Con questo fattore di accelerazione un'ora di event-time corrisponde a 0.0625s di tempo reale, un giorno a 1.5s e l'intero dataset viene riprodotto in circa 3 minuti.

Il producer può inoltre introdurre **eventi fuori ordine** ritardandone la consegna, senza modificarne il timestamp logico, e invia **marker di fine stream** per drenare le ultime finestre.

Apache Avro e Schema Registry

Gli eventi sono serializzati dal producer su Kafka in formato **Apache Avro**, un formato compatto e tipizzato, adatto allo scambio di record su uno stream. Avro è un tipo di formato basato su uno **schema** che definisce la tipizzazione: nel nostro caso lo schema **flight.avsc**.

La gestione dell'evoluzione dello schema è gestita in maniera **centralizzata** tramite un **Confluent Schema Registry**, un servizio che memorizza le versioni degli schemi e assegna a ciascuno un identificativo numerico.

Questo formato consente al consumer di risolvere lo schema tramite il registry e decodificare il record senza doverlo trasmettere a ogni messaggio.

I layer di elaborazione leggono direttamente da questo formato, risolvendo lo schema direttamente dal registry:

- Il connettore **avro-confluent** di Flink
- La funzione **from_avro** di Spark

Iniezione del disordine

Per valutare l'impatto della completezza dei risultati sulla latenza del livello di processamento in presenza di eventi fuori ordine, il producer può introdurre **disordine** nell'iniezione degli eventi, generando **eventi fuori ordine** (*out-of-order*).

Il disordine è *simulato* **ritardando la consegna di una frazione di eventi** (*una percentuale $p=0.3$*) che viene inviato più tardi senza mai modificarne l'event time, comparando nel log Kafka dopo eventi a lui successivi.

Operativamente un evento viene posto in una heapq ordinata per istante di consegna pianificato (in ritardo rispetto all'event time originale) e rilasciato solo quando il tempo di replay raggiunge tale istante.

L'entità del ritardo è campionata da una distribuzione uniforme con un tetto massimo definito che rappresenta il **bound di out-of-orderness** D con cui andrà poi a confrontare il watermark.

Marker di fine stream

- Lo stream ha una **dimensione finita**, trattandosi di un ***replay*** di un dataset costruito precedentemente.
- L'ultima finestra event time non verrebbe chiusa spontaneamente in assenza di nuovi eventi che ne facciano avanzare il watermark.
- Per drenarla in modo automatico, al termine del replay il **producer** emette su ogni partizione del topic un **marker di fine stream (evento sentinella)**.
 - Il marker di fine stream ha event time collocato *“molto nel futuro”* rispetto all'anno di riferimento del dataset: abbiamo optato per l'anno 2200.

Ciò spinge tutti i watermark per-partizione oltre l'ultima finestra reale, che viene così calcolata ed emessa senza ricorrere al comando manuale di drenaggio di Flink

Il marker è identificato dallo speciale *codice compagnia* `__EOS__` ed è quindi escluso da tutte le query.

Messaging Layer: Apache Kafka

Il trasporto degli eventi è affidato ad Apache Kafka, un sistema di messaggistica distribuito basato su log append-only e partizionato.

Il cluster è composto **da nodi** in modalità KRaft:

- Un nodo assume il ruolo di **Leader Broker**
- Il secondo nodo assume il ruolo di **Follower Broker**
 - Abilita la replicazione con **fattore di replicazione** pari a **2**

Gli eventi sono pubblicati sul topic **flights**, creato con **4 partizioni**.

Il partizionamento abilita il consumo parallelo da parte dei task di Flink e Spark.

Kafka UI, un servizio ausiliario, è incluso per il monitoraggio e il debugging del cluster.

Processing Layer: Apache Flink

Apache Flink è il framework di **Data Stream Processing (DSP)** utilizzato.

Nella nostra architettura abbiamo **un cluster Flink** composto da:

- Un **JobManager (Master)**
- Due **TaskManager (Worker)** (*processi*) con **livello di parallelismo** totale di default pari a **4**, quindi **4 TaskSlots** (*thread*) in totale.
 - Il numero di TaskManager e il livello di parallelismo sono parametri configurabili per l'esecuzione dei diversi esperimenti.

Il TaskManager esegue le tre query: i job Flink consumano il topic Kafka, assegnano timestamp e watermark in event time, calcolano aggregazioni su finestre temporali e rendono lo stato tollerante ai guasti tramite checkpointing.

Processing Layer: Apache Flink

Altri aspetti da considerare in **Flink** nel nostro studio:

- **L'intervallo di emissione periodica del watermark** (auto-watermark-interval) è impostato a **1ms** (rispetto ai 200ms di default), per rendere trascurabile il gap virtuale introdotto e isolare l'effetto di d , negli esperimenti.
 - Per gli operatori Python, la durata del bundle è impostata **50ms** (rispetto a 1s di default)
- **Il numero di eventi scartati perché arrivati oltre il watermark** è misurato direttamente dalla metrica Flink numLateRecordsDropped, raccolta dai job attraverso REST API.
- Semantica **EXACTLY-ONCE**
- **Restart strategy** in caso di guasti con 10 tentativi massimi, con 10s di attesa ciascuno

InfluxDB, TimescaleDB e Grafana

L'output certificato viene scritto in formato CSV. In parallelo, gli stessi risultati alimentano la dashboard:

- Verso **InfluxDB** tramite topic Kafka dedicati e **Telegraf**
 - InfluxDB è un database time-series nativo
 - Telegraf agisce come *ponte* per l'instradamento dei dati tra un topic di output di Kafka e InfluxDB. Si tratta di un agente il cui scopo principale è raccogliere, elaborare e inviare metriche ed eventi.
- Verso **TimescaleDB** tramite sink JDBC *nativo di Flink*.
 - TimescaleDB è un database relazionale SQL: PostgreSQL con estensione time-series

Grafana legge i dati da InfluxDB e TimescaleDB e mostra l'andamento temporale delle metriche prodotte delle query.

Esperimenti

Per far fronte agli obiettivi, sono stati valutati sperimentalmente i seguenti aspetti:

- L'impatto del **parallelismo** dell'applicazione a fronte della latenza di processamento
- L'impatto della **completezza** dei risultati rispetto alla **latenza** introducendo out-of-orderness, dove il rilevamento degli eventi fuori ordine viene gestito tramite **watermarking**
- Confronto delle prestazioni tra **Apache Flink** e **Apache Spark Structured Streaming** nell'esecuzione delle tre query

Query e Risultati

Query - Introduzione

Per ciascuna delle query che andremo a vedere sono richieste osservazioni su **finestre di varie dimensioni**. In particolare:

- **Q1:** *1 ora*
- **Q2:** *1 ora, 6 ora, Finestra Globale*
- **Q3:** *1 giorno, 7 giorni, Finestra Globale*

Query 1 - Introduzione

Monitoraggio real time dello stato operativo delle compagnie aeree AA, DL, UA, WN

Per ogni compagnia, per finestre di 1 ora calcolare:

- Numero totale dei voli
- Numero voli cancellati
- Valor medio di ritardo in partenza
- Rateo di cancellazione

Query 1 - Flink

```
CREATE TABLE flights (  
    event_time  BIGINT,  
    airline     STRING,  
    dep_delay   DOUBLE,  
    cancelled   DOUBLE,  
    diverted    DOUBLE,  
    rowtime     AS TO_TIMESTAMP_LTZ(event_time, 3),  
    WATERMARK FOR rowtime AS rowtime - INTERVAL '0' SECOND ?????  
) WITH (  
    'connector'           = 'kafka',  
    'topic'               = 'flights',  
    'properties.bootstrap.servers' = 'kafka:9092',  
    'properties.group.id'  = 'flink-q1',  
    'scan.startup.mode'    = 'earliest-offset',  
    'format'               = 'avro-confluent',  
    'avro-confluent.schema-registry.url' = 'http://schema-registry:8081',  
    'avro-confluent.schema-registry.subject' = 'flights-value'  
)
```

Query 1

```
-- Solo le 4 compagnie target
CREATE TEMPORARY VIEW flights_q1 AS
SELECT * FROM flights
WHERE airline IN ('AA', 'DL', 'UA', 'WN');
```

```
-- Aggregazione oraria per compagnia
```

```
SELECT
  window_start,
  window_end,
  airline,

  COUNT(*) AS num_flights,
  COUNT(*) FILTER (WHERE COALESCE(cancelled, 0.0) < 0.5
                    AND COALESCE(diverted, 0.0) < 0.5) AS completed,
  COUNT(*) FILTER (WHERE COALESCE(cancelled, 0.0) >= 0.5) AS cancelled,
  COUNT(*) FILTER (WHERE COALESCE(diverted, 0.0) >= 0.5) AS diverted,
  -- Ritardo medio in partenza (solo voli non cancellati)
  AVG(dep_delay) FILTER (WHERE COALESCE(cancelled, 0.0) < 0.5) AS dep_delay_mean,
  -- % cancellati sul totale
  CAST(COUNT(*) FILTER (WHERE COALESCE(cancelled, 0.0) >= 0.5) AS DOUBLE)
    * 100.0 / COUNT(*) AS cancellation_rate,
  -- % voli non cancellati con ritardo > 15 min
  CAST(COUNT(*) FILTER (WHERE COALESCE(cancelled, 0.0) < 0.5 AND dep_delay > 15) AS DOUBLE)
    * 100.0
    / NULLIF(COUNT(*) FILTER (WHERE COALESCE(cancelled, 0.0) < 0.5), 0)
    AS late_departure_rate
FROM TABLE(TUMBLE(TABLE flights_q1, DESCRIPTOR(rowtime), INTERVAL '1' HOUR))
GROUP BY window_start, window_end, airline
```

Query 1 - Spark - Prefiltro

```
flights = (  
    read_flights_stream(spark, runtime_cfg, "q1")  
    .withWatermark("rowtime", f"{watermark_delay} seconds")  
)  
AIRLINES = ["AA", "DL", "UA", "WN"]  
is_target_airline = F.col("airline").isin(AIRLINES)  
q1_input = flights.filter(is_target_airline)
```

Query 1 - Spark

```
aggregated = (  
    q1_input.groupBy(F.window("rowtime", "1 hour").alias("w"), F.col("airline"))  
    .agg(  
        F.sum(F.when(is_real, 1).otherwise(0)).cast("long").alias("num_flights"),  
        F.sum(F.when(is_real & is_not_cancelled & is_not_diverted, 1).otherwise(0))  
            .cast("long").alias("completed"),  
        F.sum(F.when(is_real & ~is_not_cancelled, 1).otherwise(0))  
            .cast("long").alias("cancelled"),  
        F.sum(F.when(is_real & ~is_not_diverted, 1).otherwise(0))  
            .cast("long").alias("diverted"),  
        # Ritardo medio in partenza (solo voli non cancellati)  
        F.avg(F.when(is_real & is_not_cancelled, F.col("dep_delay")))  
            .alias("dep_delay_mean"),  
        # Contatori di supporto per i rate  
        F.sum(F.when(is_real & is_not_cancelled, 1).otherwise(0))  
            .cast("long").alias("_non_cancelled"),  
        F.sum(F.when(is_real & is_not_cancelled & (F.col("dep_delay") > 15.0), 1).otherwise(0))  
            .cast("long").alias("_late_departures"),  
    )  
)
```

Query 1 - Spark

```
.filter(F.col("num_flights") > 0)
# % cancellati sul totale
.withColumn(
    "cancellation_rate",
    F.col("cancelled").cast("double") * F.lit(100.0) / F.col("num_flights"),
)
# % voli non cancellati con ritardo > 15 min
.withColumn(
    "late_departure_rate",
    F.when(
        F.col("_non_cancelled") > 0,
        F.col("_late_departures").cast("double") * F.lit(100.0) / F.col("_non_cancelled"),
    ),
)
)
```

Query 2 - Introduzione

Calcolare per ogni finestra considerando solo gli aeroporti con almeno 30 voli non cancellati e non deviati, la classifica dei 10 aeroporti con meno ritardi significativi in partenza, per ciascuno di essi riportare:

- Totale voli nella finestra
- Numero voli con ritardo in partenza
- Valor medio di ritardo in partenza
- Massimo DEP_DELAY
- Lista di al più 20 voli con ritardo più significativo

Le finestre considerate per questa query sono di 1 ora, 6 ore ed una finestra globale dall'inizio del dataset

Query 2 - Flink - Filtro

Dopo aver creato in prima base la vista in SQL, ne apriamo una seconda per il filtraggio delle solo righe utili allo svolgimento della query stessa

```
t_env.execute_sql("""  
    CREATE TEMPORARY VIEW completed_flights AS  
    SELECT * FROM flights  
    WHERE COALESCE(cancelled, 0.0) < 0.5  
    AND COALESCE(diverted, 0.0) < 0.5 """)
```

Query 2 - Flink - Aggregazione

A questo punto lo step è proprio quello di definire le query stesse che il job Flink dovrà andare ad eseguire.

```
stats_view = f"q2_stats_{label}"
t_env.execute_sql(f"""
    CREATE TEMPORARY VIEW {stats_view} AS
    {make_stats_view_sql(window_sql)}""")

ranked_sql = make_topn_sql(stats_view)
```

Ma vediamo la definizione di `make_stats_view_sql()` nella prossima slide

Query 2 - Flink - Aggregazione

```
def make_stats_view_sql(window_interval: str) -> str:
    return f"""
        SELECT
            window_start,
            window_end,
            origin_airport_id,
            COUNT(*)                                AS num_flights,
            COUNT(*) FILTER (WHERE dep_delay > 30.0) AS severe_delays,
            AVG(dep_delay)                          AS dep_delay_mean,
            MAX(dep_delay)                          AS dep_delay_max,
            ARRAY_AGG(CAST(ROW(dep_delay, airline, dest_airport_id) AS
                           ROW<dep_delay DOUBLE, carrier STRING, dest INT>)
                       ) FILTER (WHERE dep_delay > 30.0) AS severe_flights
        FROM TABLE(TUMBLE(TABLE completed_flights, DESCRIPTOR(rowtime),
                           {window_interval}))
        GROUP BY window_start, window_end, origin_airport_id
        HAVING COUNT(*) >= 30
    """
```

Query 2 - Ranking

```
def make_topn_sql(stats_view: str, top_n_airports: int = 10) -> str:
    return f"""
        SELECT
            window_start                                AS ts,
            rn                                           AS airport_rank,
            origin_airport_id,
            num_flights,
            severe_delays,
            dep_delay_mean,
            dep_delay_max,
            format_top_delayed(severe_flights) AS delayed_flights
        FROM (
            SELECT *,
                ROW_NUMBER() OVER (
                    PARTITION BY window_start, window_end
                    ORDER BY severe_delays DESC, dep_delay_mean DESC, origin_airport_id ASC
                ) AS rn
            FROM {stats_view}
        )
        WHERE rn <= {top_n_airports}
    """
```

Query 2

```
@udf(result_type=DataTypes.STRING(), input_types=[DataTypes.ARRAY(SEVERE_ROW)])
def format_top_delayed(flights) -> str:

    if not flights:
        return "[]"
    def sort_key(row):
        delay = row[0] if row[0] is not None else float("-inf")
        carrier = row[1] or ""
        dest = 0 if row[2] is None else row[2]
        return (-delay, carrier, dest)
    #formattazione carrier,dest,ritardo
    top = sorted(flights, key=sort_key)[:TOP_N]
    return "[" + ", ".join(
        f"({r[1]}, {0 if r[2] is None else r[2]}, {r[0]:.2f})" for r in top
    ) + "]"
```

Query 2 - Flink - Finestra Globale

Adesso vediamo l'approccio utilizzato per la finestra globale in Datastream API

```
result_ds = (  
    t_env.to_data_stream(t_env.from_path("flights")) #conversione in datastream  
    .assign_timestamps_and_watermarks(watermark_strategy)  
    .filter(is_q2_global_event) # funzione presa da global_window_ops  
    .map(project_q2_global_event)  
    .key_by(lambda _: "q2-global")  
    .window(GlobalWindows.create())  
    .trigger(ContinuousEventTimeTrigger.of(Time.days(1))) #ogni quanto  
aggreghiamo  
    .aggregate(Q2GlobalAggregate(),  
               window_function=Q2GlobalWindowFunction())  
)
```

Query 2 - Flink - Finestra Globale

```
def is_q2_global_event(value) -> bool:
    airline = value[1]
    cancelled = value[5] or 0.0
    diverted = value[6] or 0.0
    return (
        airline != EOS_AIRLINE
        and cancelled < 0.5
        and diverted < 0.5
        and value[2] is not None #origin airport valido
    )

def project_q2_global_event(value):
    # (event_time, origin_airport_id, dest_airport_id, dep_delay, carrier)
    return (value[0], value[2], value[3], value[4], value[1])
```

Query 2 - Flink -Finestra Globale

Una volta definita la finestra globale tramite Datastream API, utilizziamo per essa la seguente funzione di aggregazione sui singoli eventi:

```
class Q2GlobalAggregate(AggregateFunction):
    def create_accumulator(self):
        return {"airports": {}}          # un'entry indipendente per aeroporto

    def add(self, value, acc):            # chiamata per OGNI volo
        state = acc["airports"].setdefault(airport_id, _new_airport_state())
        state["num_flights"] += 1
        if dep_delay is not None:
            state["mean_count"] += 1
            state["mean"] += (dep_delay - state["mean"]) / state["mean_count"]
            # media online (Welford)
            state["dep_delay_max"] = max(state["dep_delay_max"], dep_delay)
            if dep_delay > 30.0:
                state["severe_delays"] += 1
                _update_top20(state["top20"], dep_delay, airline, dest)      # heap

top-20
return acc
```

Query 2 - Finestra Globale

Ad ogni scatto giornaliero viene prodotto il ranking parziale:

```
class Q2GlobalWindowFunction(ProcessWindowFunction):
    def process(self, key, context, elements):
        acc = next(iter(elements))
        yield from _ranked_rows(acc["airports"], context.current_watermark())

def _ranked_rows(airports, watermark_ms):
    ranked = [(aid, s) for aid, s in airports.items() if s["num_flights"] >= 30]
    # soglia ≥30
    ranked.sort(key=lambda it: (-it[1]["severe_delays"], -it[1]["mean"], it[0]))
    # stesso ordinamento
    for rank, (aid, s) in enumerate(ranked[:10], start=1):
        # top-10
        yield Row(snapshot_ts, rank, aid, s["num_flights"], s["severe_delays"],
float(state["mean"]), float(state["dep_delay_max"] or 0.0),
_format_top_delayed(state["top20"]), )
```

Query 2 - Spark

Tiene solo i voli realmente partiti (non cancellati, non dirottati):

```
def build_stats(flights, window_duration):  
    completed = flights.filter(not_cancelled() & not_diverted())  
    severe = F.col("dep_delay") > 30.0  
  
    return (  
        #definizione della window  
        completed.groupBy(F.window("rowtime", window_duration), "origin_airport_id")  
        .agg(  
            F.count("*").alias("num_flights"),  
            F.sum(F.when(severe, 1).otherwise(0)).alias("severe_delays"),  
            F.avg("dep_delay").alias("dep_delay_mean"),  
            F.max("dep_delay").alias("dep_delay_max"),  
            F.collect_list(F.when(severe, F.struct("airline", "dest_airport_id",  
"dep_delay"))).alias("top20"),  
        )  
        .filter(F.col("num_flights") >= 30)  
    )
```


Query 2 - Spark

Factory che fa effettivamente la classifica sui minibatch ricevuti:

```
def ranked_batch_writer(output_path):  
    def write(batch_df, batch_id):  
        #dataframe con i risultati  
        if batch_df.rdd.isEmpty():  
            return  
        #partition by per fornire classifica per finestra  
        rank_window = Window.partitionBy("ts").orderBy(  
            F.desc("severe_delays"),  
            F.desc("dep_delay_mean"),  
            F.asc("origin_airport_id"),  
        )  
  
        ranked = (  
            batch_df.withColumn("rank", F.row_number().over(rank_window))  
            .filter(F.col("rank") <= 10)  
            .select(  
                "ts", "rank", "origin_airport_id", "num_flights",  
                "severe_delays", "dep_delay_mean", "dep_delay_max", "delayed_flights",  
            )  
        )  
        #forza 1 file solo invece che n partizione 1 solo file in uscita  
        ranked.coalesce(1).write.mode("append").option("header", "false").csv(output_path)  
    return write
```

Query 2 - Spark - Finestra Globale

Analoga alla build global window, facciamo uno snapshot aggiornando la top 10 aeroporti:

```
def build_q2_global_snapshots(flights, *, watermark_delay_seconds):
    is_eos = F.col("airline") == F.lit(EOS_AIRLINE)
    is_real = (~is_eos & not_cancelled() & not_diverted()
              & F.col("origin_airport_id").isNotNull())

    relevant = (
        flights.withColumn("global_rowtime", _raw_event_time())           # timestamp evento
                .withWatermark("global_rowtime", f"{watermark_delay_seconds} seconds") # watermark proprio
                .filter(is_real | is_eos)
                .withColumn("_global_key",                                  # chiave (EOS -1)
                           F.when(is_eos, F.lit(-1)).otherwise(F.col("origin_airport_id")))
                .select("_global_key", "global_rowtime", "event_time",
                       "airline", "dest_airport_id", "dep_delay")
    )
    #Crea la chiave di partizionamento ogni aereoporto una chiave
    return relevant.groupBy("_global_key").applyInPandasWithState( #per definire proc arbitrario
        _q2_state_function, # la funzione che aggiorna lo stato ed emette righe
        outputStructType=Q2_GLOBAL_OUTPUT_SCHEMA, # schema delle righe emesse
        stateStructType=GLOBAL_STATE_SCHEMA, # schema dello stato salvato
        outputMode="Append",
        timeoutConf=GroupStateTimeout.EventTimeTimeout,
    )
```

Query 2 - Spark

Funzione per accumulare ed emettere gli snapshot:

```
def _q2_state_function(key, pdf_iter, state): #una volta per chiave, quindi per aereoporto in ogni micro batch
    airport_id = int(key[0])
    if state.exists:
        stored, next_snapshot = pickle.loads(state.get[0]), state.get[1]
    else: #primo batch di questo aereoporto stato vuoto
        stored, next_snapshot = {"committed": _new_q2_stats(), "pending": {}}, FIRST_DAY_MS
    # accumula i voli nel corretto bucket del giorno
    for pdf in pdf_iter:
        for row in pdf.itertuples():
            bucket = _bucket_end_ms(int(row.event_time))
            target = stored["committed"] if bucket < next_snapshot \
                else stored["pending"].setdefault(bucket, _new_q2_stats())
            _add_q2_flight(target, row.dep_delay, row.airline, row.dest_airport_id)
    # a ogni giorno superato dal watermark: fondi i bucket dovuti ed emetti
    output = []
    while next_snapshot < state.getCurrentWatermarkMs():
        for b in [b for b in stored["pending"] if b <= next_snapshot]:
            _merge_q2_stats(stored["committed"], stored["pending"].pop(b))
        if stored["committed"]["num_flights"] >= 30:
            output.append(_build_snapshot_row(airport_id, stored["committed"]))
            next_snapshot += ONE_DAY
    # salva stato + prossimo confine, e arma il timer event-time
    state.update((pickle.dumps(stored), next_snapshot))
    state.setTimeoutTimestamp(next_snapshot)
    if output:
        yield pd.DataFrame(output)
```

Query 2 - Spark

Assorbe 1 volo dentro le statistiche di 1 aeroporto:

```
def _add_q2_flight(stats: dict, *, dep_delay: float | None, airline: str | None, dest_airport_id: int
| None,) -> None:
    stats["num_flights"] += 1
    if dep_delay is None:
        return

    delay = float(dep_delay)
    stats["delay_count"] += 1
    stats["delay_sum"] += delay
    stats["dep_delay_max"] = (
        delay
        if stats["dep_delay_max"] is None
        else max(stats["dep_delay_max"], delay)
    )
    if delay > 30.0:
        stats["severe_delays"] += 1
        _update_top20(stats["top20"], delay, airline, dest_airport_id)
```

Query 2 - Spark

Accumulo incrementale

```
def _merge_q2_stats(target: dict, source: dict) -> None:
    target["num_flights"] += source["num_flights"]
    target["delay_count"] += source["delay_count"]
    target["delay_sum"] += source["delay_sum"]
    target["severe_delays"] += source["severe_delays"]
    if source["dep_delay_max"] is not None:
        target["dep_delay_max"] = (
            source["dep_delay_max"]
            if target["dep_delay_max"] is None
            else max(target["dep_delay_max"], source["dep_delay_max"])
        )
    for delay, airline, destination in source["top20"]:
        _update_top20(target["top20"], delay, airline, destination)
```

Query 2 - Spark

Mantiene i 20 voli più in ritardo di un singolo aeroporto

```
def _update_top20(
    heap: list[tuple[float, str, int]],
    dep_delay: float,
    airline: str | None,
    dest_airport_id: int | None,
) -> None:
    item = (
        float(dep_delay),
        airline or "",
        0 if dest_airport_id is None else int(dest_airport_id),
    )
    if len(heap) < TOP_N_FLIGHTS:
        heapq.heappush(heap, item)
    elif item > heap[0]:
        heapq.heapreplace(heap, item)
```

Query 3 - Introduzione

Ricavare le fasce orarie di partenza di ciascun volo non cancellato delle compagnie American Airlines, Delta, United e Southwest, per ciascuna compagnia aerea e fascia oraria calcolare:

- Totale numero di voli
- DEP_DELAY minimo e massimo
- Percentili
 - 25-esimo
 - 50-esimo (mediana)
 - 75-esimo
 - 90-esimo

Le finestre considerate per questa query sono di 1 giorno, 7 giorni ed una finestra globale dall'inizio del dataset

Query 3 - Flink

```
bounded_decoded_ds = t_env.to_data_stream(t_env.from_path(bounded_source))
keyed_stream = (
    bounded_decoded_ds
    .filter(_is_q3_relevant)
    .map(_project_q3_event, output_type=Types.PICKLED_BYTE_ARRAY())
    .name("Q3Events[python]")
    .key_by(
        lambda v: group_key(v[1], v[2]),    # v[1]=airline, v[2]=hour e.g. AA|09
        key_type=Types.STRING(),
    )
)
```


Query 3 - Flink

```
AIRLINES: frozenset[str] = frozenset({"AA", "DL", "UA", "WN"})
```

```
def _is_q3_relevant(value) -> bool:
    airline      = value[1]
    dep_delay    = value[3]
    cancelled     = value[4]
    diverted      = value[5]
    return (
        airline in AIRLINES
        and (cancelled or 0.0) < 0.5
        and (diverted or 0.0) < 0.5
        and dep_delay is not None
    )

def _project_q3_event(value):
    # (event_time, airline, hour_band, dep_delay)
    return (value[0], value[1], hour_band(value[2]), float(value[3]))

def hour_band(crs_dep_time: int) -> int:
    return (crs_dep_time // 100) % 24
```

Query 3 - Flink

```
def build_window_pipeline(label, window_assigner, results_path,
                          keyed_stream, t_env, stmt_set, cfg):
    # Aggregazione a finestra con DDSketch, con side-output dei record tardivi —
    late_tag = OutputTag(f"q3-late-{label}", Types.PICKLED_BYTE_ARRAY())
    result_ds = (
        keyed_stream
        .window(window_assigner)                # 1d | 7d
        .side_output_late_data(late_tag)
        .aggregate(
            Q3AggregateFunction(cfg.sketch_alpha),
            window_function=Q3WindowFunction(),
            accumulator_type=Types.PICKLED_BYTE_ARRAY(),
            output_type=Q3_OUTPUT_TYPE,)
        .name(f"Q3Window[{label}]"))
    # Record tardivi contatore metrica (osservabilità gestione watermark) —
    (result_ds
     .get_side_output(late_tag)
     .map(Q3LateDropCounter(), output_type=Types.LONG())
     .name(f"Q3LateDrops[{label}]"))
    # Sink CSV: output certificato (ts, airline, hour, num_flights, min, p25..p90, max) —
    view = f"q3_{label}"
    t_env.create_temporary_view(view, t_env.from_data_stream(result_ds, Q3_RESULT_SCHEMA))
    t_env.execute_sql(CSV_SINK_DDL.format(name=f"q3_csv_{label}", path=results_path))
    stmt_set.add_insert_sql(f"INSERT INTO q3_csv_{label} SELECT * FROM {view}")
    logger.info("Q3 [%s] | CSV sink → %s", label, results_path)
```

Query 3 - Flink

```
def build_global_window_pipeline(keyed_stream, t_env, stmt_set, cfg):
    # Finestra cumulativa: un unico GlobalWindow con snapshot giornalieri —
    result_ds = (
        keyed_stream
        .window(GlobalWindows.create())
        .trigger(ContinuousEventTimeTrigger.of(Time.days(1))) # emette uno snapshot per giorno di
event-time
        .aggregate(
            Q3AggregateFunction(cfg.sketch_alpha),
            window_function=Q3GlobalWindowFunction(), # scarta gli snapshot vuoti
            accumulator_type=Types.PICKLED_BYTE_ARRAY(),
            output_type=Q3_OUTPUT_TYPE,
        )
        .name("Q3GlobalWindow")
    )

    # Sink CSV: risultato cumulativo dall'inizio del dataset fino a ogni giorno —
    view = "q3_global"
    t_env.create_temporary_view(view, t_env.from_data_stream(result_ds, Q3_RESULT_SCHEMA))
    t_env.execute_sql(CSV_SINK_DDL.format(name="q3_csv_global", path=cfg.results_path_global))
    stmt_set.add_insert_sql(f"INSERT INTO q3_csv_global SELECT * FROM {view}")
    logger.info("Q3 [global] | GlobalWindow CSV sink → %s", cfg.results_path_global)
```

Query 3 - Flink

Funzioni di aggregazione sugli sketch

```
class Q3AggregateFunction(AggregateFunction):
    def __init__(self, alpha: float):
        self._alpha = alpha
    def create_accumulator(self) -> Q3DDSketch:
        return Q3DDSketch(self._alpha)
    def add(self, value, acc: Q3DDSketch) -> Q3DDSketch:
        acc.add(value[3])          # value = (event_time_ms, airline, hour, dep_delay)
        return acc
    def get_result(self, acc: Q3DDSketch) -> Q3DDSketch:
        return acc
    def merge(self, acc: Q3DDSketch, other: Q3DDSketch) -> Q3DDSketch:
        acc.merge(other)
        return acc

def _q3_result_row(ts: datetime, key: str, sketch: Q3DDSketch) -> Row:
    airline, hour_str = key.split("|")
    return Row(ts, airline, int(hour_str), sketch.count, sketch.min_value,
               round(sketch.quantile(0.25), 2),
               round(sketch.quantile(0.50), 2),
               round(sketch.quantile(0.75), 2),
               round(sketch.quantile(0.90), 2),
               sketch.max_value,
    )
```

Query 3 - Flink

Ultimo step la formattazione al risultato finale

```
class Q3WindowFunction(ProcessWindowFunction):
    def process(self, key, context, elements):
        sketch = next(iter(elements))
        ts = datetime.fromtimestamp(
            context.window().start / 1000.0, tz=timezone.utc
        ).replace(tzinfo=None)
        yield _q3_result_row(ts, key, sketch)

class Q3GlobalWindowFunction(ProcessWindowFunction):
    """Uno snapshot DDSketch cumulativo per chiave e per giorno di event-time."""
    def process(self, key, context, elements):
        sketch = next(iter(elements))
        if sketch.count == 0:
            return
        yield _q3_result_row(
            _global_snapshot_ts(context.current_watermark()),
            key,
            sketch,
        )
```

Query 3 - Flink

```
DATASET_START_TS = datetime(2025, 1, 1)
DATASET_START_MS = int(DATASET_START_TS.replace(tzinfo=timezone.utc).timestamp() * 1000)
GLOBAL_SNAPSHOT_INTERVAL_MS = 86_400_000 # 1 giorno

def _global_snapshot_ts(current_watermark_ms: int) -> datetime:
    if current_watermark_ms < DATASET_START_MS:
        return DATASET_START_TS
    snapshot_ms = (
        DATASET_START_MS
        + ((current_watermark_ms - DATASET_START_MS) // GLOBAL_SNAPSHOT_INTERVAL_MS)
        * GLOBAL_SNAPSHOT_INTERVAL_MS
    )
    return datetime.fromtimestamp(snapshot_ms / 1000.0, tz=timezone.utc).replace(tzinfo=None)
```

Query 3 - Spark

Finestre 1D/7D

```
relevant.groupBy("airline", "hour").applyInPandasWithState(  
    q3_window_state_function(alpha, window_duration_ms, window_offset_ms),  
    outputStructType=Q3_GLOBAL_OUTPUT_SCHEMA,  
    stateStructType="payload binary",          # DDSketch serializzati (pickle)  
    outputMode="Append",  
    timeoutConf=GroupStateTimeout.EventTimeTimeout)
```

Finestra globale

```
relevant.groupBy("airline", "hour").applyInPandasWithState(  
    q3_state_function(alpha),  
    outputStructType=Q3_GLOBAL_OUTPUT_SCHEMA,  
    stateStructType="payload binary, next_snapshot_ms long",  
    outputMode="Append",  
    timeoutConf=GroupStateTimeout.EventTimeTimeout)
```

Query 3 - Spark

```
def q3_state_function(alpha):
    def update(key, pdf_iter, state):
        airline, hour = str(key[0]), int(key[1])
        if state.exists:
            stored, next_snapshot_ms = pickle.loads(state.get[0]), state.get[1]
        else:
            stored = {"committed": Q3DDSketch(alpha), "pending": {}}
            next_snapshot_ms = DATASET_START_MS + GLOBAL_SNAPSHOT_INTERVAL_MS
        committed, pending = stored["committed"], stored["pending"]

        # 1) ogni volo nello sketch del suo bucket giornaliero
        for pdf in pdf_iter:
            for row in pdf.itertuples(index=False):
                if row.airline == EOS_AIRLINE or airline == EOS_AIRLINE: continue
                bucket_end = _bucket_end_ms(int(row.event_time))
                target = committed if bucket_end < next_snapshot_ms \
                    else pending.setdefault(bucket_end, Q3DDSketch(alpha))
                target.add(float(row.dep_delay))
```


Query 3 - Spark

```
# 2) a ogni giorno superato dal watermark: fondi i bucket dovuti ed emetti i percentili
out = []
while airline != EOS_AIRLINE and next_snapshot_ms < int(state.getCurrentWatermarkMs()):
    for b in sorted(b for b in pending if b <= next_snapshot_ms):
        committed.merge(pending.pop(b))
    if committed.count > 0:
        out.append(_q3_result(next_snapshot_ms, airline, hour, committed))
    next_snapshot_ms += GLOBAL_SNAPSHOT_INTERVAL_MS

# 3) persisti stato + confine, arma il timer
if airline != EOS_AIRLINE:
    state.update((pickle.dumps(stored), next_snapshot_ms))
    state.setTimeoutTimestamp(next_snapshot_ms)
if out: yield pd.DataFrame(out)
return update
```

Query 3 - Spark

```
def q3_window_state_function(alpha, window_duration_ms, window_offset_ms):
    def update(key, pdf_iter, state):
        airline, hour = str(key[0]), int(key[1])
        windows = pickle.loads(state.get[0]) if state.exists else {} # {window_start: sketch}
        wm = int(state.getCurrentWatermarkMs())

        # 1) ogni volo nello sketch della sua finestra (scarta i tardivi)
        for pdf in pdf_iter:
            for row in pdf.itertuples(index=False):
                ws = _q3_window_start_ms(row.event_time, window_duration_ms, window_offset_ms)
                if wm >= 0 and ws + window_duration_ms - 1 <= wm: # finestra già chiusa → tardivo
                    continue
                windows.setdefault(ws, Q3DDSketch(alpha)).add(float(row.dep_delay))

        # 2) emetti e rimuovi le finestre mature (fine ≤ watermark)
        out = []
        for ws in sorted(w for w in windows if w + window_duration_ms - 1 <= wm):
            s = windows.pop(ws)
            if s.count > 0:
                out.append(_q3_result(ws, airline, hour, s))
```

Query 3 - Spark

```
# 3) persisti lo stato residuo e arma il timer sulla prossima chiusura
if windows:
    state.update((pickle.dumps(windows),))
    state.setTimeoutTimestamp(min(windows) + window_duration_ms - 1)
elif state.exists:
    state.remove()
if out: yield pd.DataFrame(out)
return update

def _q3_result(timestamp_ms, airline, hour, sketch):    # estrae la riga di output dallo sketch
return {"ts": _snapshot_ts(timestamp_ms), "airline": airline, "hour": hour,
        "num_flights": sketch.count, "delay_min": sketch.min_value,
        "p25": round(sketch.quantile(0.25), 2), "p50": round(sketch.quantile(0.50), 2),
        "p75": round(sketch.quantile(0.75), 2), "p90": round(sketch.quantile(0.90), 2),
        "delay_max": sketch.max_value}
```

Esperimenti:

- *Completezza vs Latenza*
- *Checkpointing*
- *Parallelismo Flink*
- *Confronto con Structured Spark Streaming*

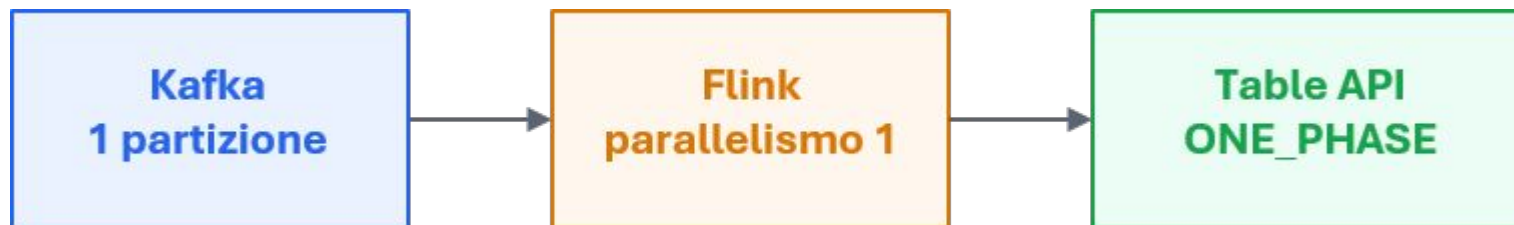
Completezza vs Latenza

Obiettivo: valutare quanti record vengono scartati come late al variare del watermark delay, in presenza di eventi fuori ordine

Generazione dell'out-of-orderness:

- Il **30% degli eventi** viene trattenuto dal producer
- Il ritardo è estratto uniformemente nell'intervallo $[0, W]$
- W è la dimensione della finestra
- L'event time originale resta invariato

Configurazione controllata, non prestazionale: adottata per ottenere una misura deterministica di record scartati



Completezza vs Latenza

Regimi di watermark e risultati

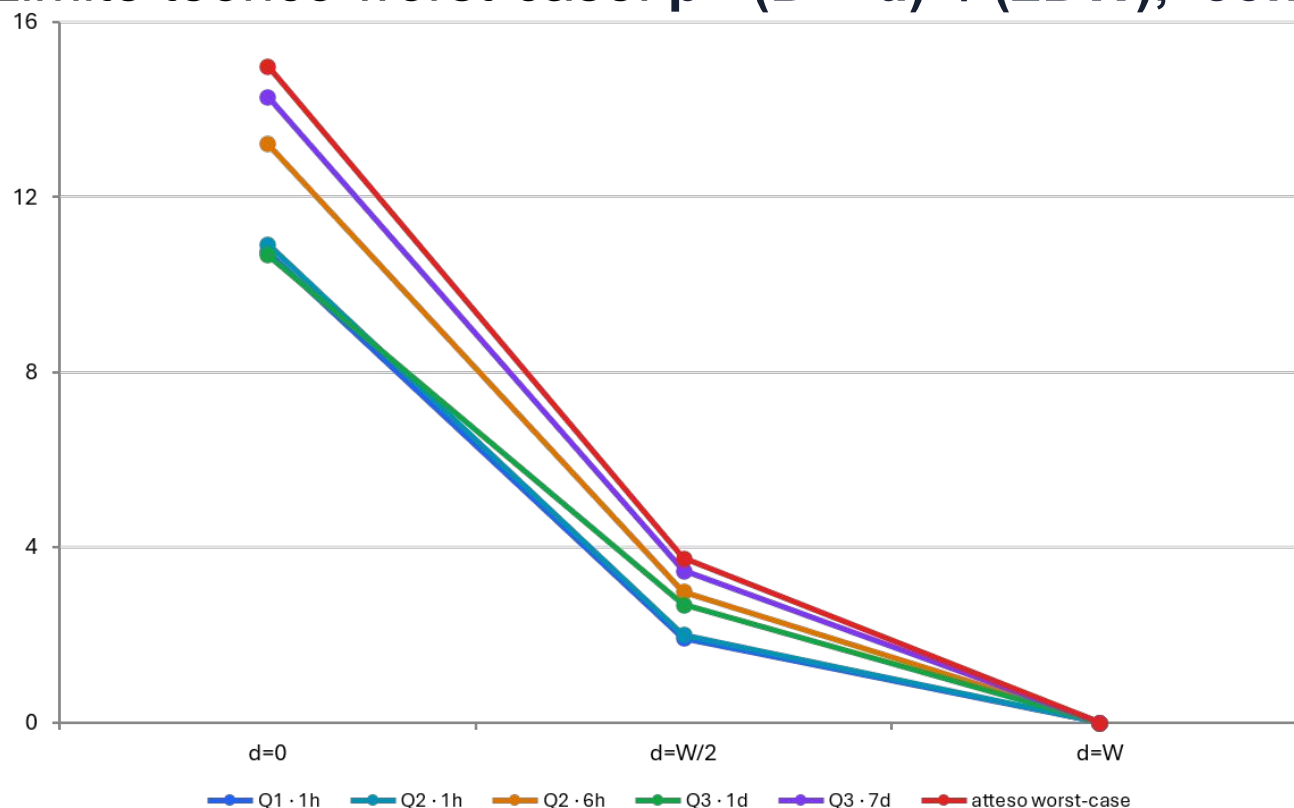
- **d=0** → Aggressivo: massima tempestività, più record scartati
- **d=W/2** → Intermedio: recupera gran parte dei tardivi
- **d=W** → Completo: copre il bound di out-of-orderness

Query	W	d [s] (event-time)	Regime	LateDropped [record]	Record persi sulla popolazione di riferimento [%]	Record persi attesi [%]
Q1	1h	0	aggressivo	142 249	10.76	15.00
		1800	intermedio	25 516	1.93	3.75
		3600	completo	0	0.00	0.00
Q2	1h	0	aggressivo	239 233	10.93	15.00
		1800	intermedio	43 941	2.01	3.75
		3600	completo	0	0.00	0.00
Q2	6h	0	aggressivo	289 648	13.23	15.00
		10800	intermedio	65 466	2.99	3.75
		21600	completo	0	0.00	0.00
Q3	1d	0	aggressivo	139 373	10.69	15.00
		43200	intermedio	35 005	2.69	3.75
		86400	completo	0	0.00	0.00
Q3	7d	0	aggressivo	186 382	14.30	15.00
		302400	intermedio	45 225	3.47	3.75
		604800	completo	0	0.00	0.00

Completezza vs Latenza

Risultati: Impatto del watermark delay

Limite teorico worst-case: $p \cdot (D - d)^2 / (2DW)$, con $D = W$



10,69–14,30%
persi con $d=0$

1,93–3,47%
persi con $d=W/2$

0%
persi con $d=W$

Il prezzo della completezza si paga quindi in latenza di emissione e nel mantenimento più lungo dello stato, non in una pausa dell'elaborazione

Checkpointing

Obiettivo: *verificare il corretto recupero dello stato dopo la perdita di un TaskManager e accertare l'assenza di record persi, duplicati o alterati*

Configurazione:

- Checkpoint ogni **10 s**
- Semantica **Exactly-Once**
- Barrier checkpoint **allineate**
- Checkpoint externalizzati
- Q1: **p=4**, 1 TaskManager
- Restart fisso job: **10 s**
- Producer accelerato a **57.600×**
- **10 run con guasto + 10 run senza guasto**

Checkpointing

Sequenza della verifica



Tempo end-to-end: dal lancio del producer al consolidamento delle finestre finali.

Producer e Kafka restano attivi durante il guasto

Checkpointing

Recovery verificato in 10/10 esecuzioni, con output identico alla baseline

Composizione del recovery

Rilevamento guasto

Ripristino

40,27 s

11,10 s

Verifica Exactly-Once

Recovery totale: 51,36 s

10/10

esecuzioni recuperate
correttamente

9.765 = 9.765

chiavi con e senza guasto

0

duplicati, mancanti e
differenze

Output canonico identico alla baseline senza guasto

Durante il guasto aumenta il consumer lag; dopo il restore Flink lo riassorbe

Checkpointing

Impatto sul tempo end-to-end

Senza guasto

209,363 s

Con guasto

209,743 s

+0,38 s $\approx$ +0,18%

51,36 s di recovery, ma solo +0,38 s sul tempo end-to-end

Conclusione: il guasto interrompe temporaneamente l'elaborazione, ma non altera i risultati; il consumer lag viene recuperato prima della conclusione del replay

Parallelismo

Obiettivo: *valutare come throughput e tempo di elaborazione cambiano aumentando il parallelismo degli operatori Flink*

Configurazione:

- Record precaricati su Kafka: stesso backlog per tutte le configurazioni
- Topic Kafka con **4 partizioni**
- Producer terminato prima dell'esecuzione: separare effetto parallelismo dal limite di ingestione del producer
- Nessun out-of-orderness

Parallelismo	Scale-up	Scale-out
p = 1	1 TM × 1 slot	—
p = 2	1 TM × 2 slot	2 TM × 1 slot
p = 4	1 TM × 4 slot	2 TM × 2 slot

Parallelismo – 1 TaskManager

Lo speedup misura guadagno rispetto a configurazione con parallelismo 1

Query	p	Throughput [rec/s]	Elapsed [s]	Busy [%]	Speedup	
Q1 (1h)	1	284.763	7,829	30,3%	1,00×	beneficio evidente a p=4; busy basso
	2	279.878	7,966	23,5%	0,98×	
	4	528.423	4,219	28,6%	1,86×	
Q2 (1h)	1	207.102	10,765	47,4%	1,00×	crescita moderata; busy resta attorno al 40–47%
	2	273.288	8,158	40,3%	1,32×	
	4	308.936	7,217	41,4%	1,49×	
Q3 (1d)	1	10.383	214,721	95,9%	1,00×	busy >90%; speedup 3,50× a p=4 → compute-bound
	2	19.830	112,431	92,8%	1,91×	
	4	36.377	61,287	90,2%	3,50×	

Q3 scala meglio perché resta compute-bound; Q1 e Q2 sono ingestion bound e quindi migliorano meno perché gli operatori non sono saturi

Parallelismo – Scale-up vs Scale-out

Lo speedup misura guadagno rispetto a configurazione con parallelismo 1

p = 2

Query	Throughput 1 TM → 2 TM	ΔThr	Elapsed [s] 1 TM → 2 TM	Δt	Busy [%] 1 TM → 2 TM	Speedup 1 TM → 2 TM
Q1	279.878 → 259.922	-7,1%	7,966 → 8,577	+7,7%	23,5 → 40,2	0,98× → 0,91×
Q2	273.288 → 263.283	-3,7%	8,158 → 8,468	+3,8%	40,3 → 42,8	1,32× → 1,27×
Q3	19.830 → 19.644	-0,9%	112,431 → 113,494	+0,9%	92,8 → 93,0	1,91× → 1,89×

p = 4

Query	Throughput 1 TM → 2 TM	ΔThr	Elapsed [s] 1 TM → 2 TM	Δt	Busy [%] 1 TM → 2 TM	Speedup 1 TM → 2 TM
Q1	528.423 → 477.414	-9,7%	4,219 → 4,670	+10,7%	28,6 → 42,1	1,86× → 1,68×
Q2	308.936 → 276.966	-10,3%	7,217 → 8,050	+11,5%	41,4 → 39,3	1,49× → 1,34×
Q3	36.377 → 32.007	-12,0%	61,287 → 69,656	+13,7%	90,2 → 89,1	3,50× → 3,08×

Passando da 1 a 2 TaskManager, il throughput diminuisce e l'elapsed aumenta; anche lo speedup rispetto alla baseline peggiora in tutte le query

Confronto con Spark

Obiettivo: *confrontare le prestazioni dei due motori a parità di dati, parallelismo e risultati prodotti*

Configurazione:

- Stesso backlog pre-caricato su Kafka
- Topic Kafka con **4 partizioni**
- Parallelismo $p=4$
- **2 TaskManager** per Flink
- Configurazione equivalente per Spark
- Stesso DDSketch e stessa accuratezza per Q3
- Throughput calcolato sul tempo di sola elaborazione

Le implementazioni sono state validate sullo stesso output; il confronto seguente riguarda esclusivamente le prestazioni

Confronto con Spark

Query	Flink throughput	Spark throughput	Flink elapsed	Spark elapsed	Risultato
Q1(1h)	477.400 rec/s	273.419 rec/s	4,670 s	8,154 s	Flink +75%
Q2(1h)	276.951 rec/s	174.818 rec/s	8,050 s	12,753 s	Flink +58%
Q3(1d)	32.007 rec/s	167.138 rec/s	69,656 s	13,339 s	Spark 5,2×

- **Q1 e Q2:** Flink è più rapido perché le query sono eseguite nativamente in JVM e Spark paga l'overhead dei micro-batch
- **Q3:** Spark è circa **5,2×** più veloce perché PyFlink paga maggiormente il trasferimento per-record tra JVM e processo Python. Spark ammortizza meglio.



TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

GRAZIE PER L'ATTENZIONE!